🛠️ HomeServe - On-Demand Home Services Booking Platform

Comprehensive Academic Review & System Architecture Documentation

> **Live Deployment Links:**
>
> - 🌐 **Production Web Application:** https://homeserve-ui.onrender.com
> - ⚙️ **Backend API & Swagger Docs:** https://homeserve-backend-ss0m.onrender.com/api/swagger-ui/index.html
> - 💻 **GitHub Repository:** https://github.com/abhinavt77/homeserve

1. Executive Summary & Project Objectives

HomeServe is an end-to-end, full-stack, on-demand home services booking platform designed to connect urban residential customers with verified, hyperlocal service professionals (electricians, plumbers, cleaners, carpenters, and appliance technicians) in real time.

Core Engineering Objectives

1. **Hyperlocal Provider Discovery:** Calculate geographic proximity using spherical trigonometry (Haversine formula & PostGIS) to identify active, available service technicians within a configurable radial threshold.
2. **Real-Time Push Dispatch:** Deliver live job requests to eligible providers with low-latency bidirectional WebSockets (STOMP over SockJS), eliminating polling overhead.
3. **Concurrency Control & Race Condition Prevention:** Enforce strict optimistic concurrency control (`@Version` attribute) at the database layer to ensure that two simultaneous provider acceptances never result in a double-booked service request.
4. **Resilient Financial Transactions:** Integrate Razorpay test-mode payment gateway with cryptographically verified HMAC-SHA256 signatures, alongside Cash on Delivery (COD) settlement.

5. **Production Cloud Deployment:** Package the layered monolith into a multi-stage Docker container deployed on Render cloud infrastructure with zero-downtime CDN edge delivery.

2. System Architecture & High-Level Design

HomeServe follows a **Strict Layered Monolith Architecture**. Microservices were intentionally avoided to minimize distributed transaction overhead, network latency, and operational complexity, while maximizing data consistency and transaction integrity.

Architectural Principles

- **Separation of Concerns:** Controllers handle HTTP parameter validation and status mapping; Services execute business rules and transactional boundaries; Repositories manage database queries.
- **DTO-Driven Communication:** Domain entities are never exposed directly to the REST interface. All contracts are enforced with strongly typed Request/Response DTOs and Jakarta Validation constraints.
- **Stateless Security:** Client sessions are entirely stateless; authentications utilize compact, cryptographically signed JSON Web Tokens (HMAC-SHA256) valid for 24 hours.

3. Technology Stack & Component Justification

Layer / Component	Technology Selected	Justification & Architectural Benefit
Backend Framework	Java 17+, Spring Boot 3.3.4	Enterprise-grade type safety, production-ready dependency injection, out-of-the-box actuator metrics, and seamless integration with JPA and STOMP messaging.
Data Persistence	Spring Data JPA / Hibernate	High-productivity ORM with native transaction management (`@Transactional`) and automatic schema migration compatibility.
Database	PostgreSQL + PostGIS (H2 Fallback)	Industry-standard relational integrity with native spatial indexing (R-Tree) for ultra-fast geographical radial queries.
Real-Time Transport	Spring WebSocket + STOMP + SockJS	STOMP provides pub/sub destination routing (`/topic` , `/queue`) over persistent bidirectional WebSocket channels, with SockJS HTTP-streaming fallback for restrictive networks.
Frontend Framework	React 18 (Vite Bundler)	Component-based UI with sub-second Hot Module Replacement (HMR), optimized Rollup production asset chunking, and declarative UI state.
Styling & Icons	Tailwind CSS + Lucide React	Utility-first responsive design, zero runtime stylesheet injection, fully accessible dark/light mode toggle.
Geospatial UI	Leaflet + OpenStreetMap (`react-leaflet`)	Completely open-source interactive mapping with zero API license fees (eliminates Google Maps billing requirements).
Security & Auth	Spring Security 6 + `jjwt 0.12.6`	Stateless JWT authentication filter pipeline with granular Method-Level Role-Based Access Control (`@PreAuthorize`).
API Documentation	Springdoc OpenAPI 2.6.0 (Swagger 3)	Interactive, self-documenting REST contract interface directly synchronized with backend annotations.
Payment Gateway	Razorpay SDK (Test Mode)	Comprehensive Indian payment rails simulation (UPI, Cards, NetBanking) with SHA256 cryptographic signature validation.
DevOps & Packaging	Multi-stage Docker + Render Cloud	Reproducible containerized build environment with non-root Linux runtime (`appuser`), automated CDN routing, and free-tier cloud hosting.

4. Database Schema & Entity Relationship (ER) Diagram

The relational data model consists of 9 core entities mapped with strict referential integrity, cascading actions, and database indexes.

Key Database Design Highlights:

1. **Concurrency Control Column (`version`):** Mapped with Hibernate `@Version` on `service_requests`. Any race condition where multiple providers attempt to accept the same job triggers an `OptimisticLockException`, ensuring atomic single-winner acceptance.
2. **Spatial Coordinate Indexing:** The `latitude` and `longitude` fields in `provider_profiles` and `addresses` are indexed for fast spherical distance calculations.
3. **Immutable Tracking Number:** Every booking is assigned a unique alphanumeric tracking code (e.g., `HS-9B4E7A21`) for public status lookup and audit tracking.

5. Core Business Flow & Concurrency Mechanics

The Step-by-Step Dispatch Sequence

The Optimistic Concurrency Algorithm

To prevent the classic double-booking problem without blocking the database table with heavy pessimistic row locks, the service employs JPA's `@Version` mechanism:

```
java
@Transactional
public BookingResponse acceptJob(Long bookingId, Long providerUserId) {
    ServiceRequest booking = bookingRepository.findById(bookingId)
            .orElseThrow(() -> new ResourceNotFoundException("Booking not found"));

    if (booking.getStatus() != RequestStatus.PENDING) {
        throw new IllegalStateException("Booking has already been claimed or cancelled");
    }

    ProviderProfile provider = providerRepository.findByUserId(providerUserId)
            .orElseThrow(() -> new ResourceNotFoundException("Provider profile not
found"));

    booking.setProvider(provider);
    booking.setStatus(RequestStatus.ACCEPTED);
    // Hibernate automatically executes:
    // UPDATE service_requests SET status='ACCEPTED', version=version+1 WHERE id=? AND
version=?
    return bookingMapper.toResponse(bookingRepository.save(booking));
}
```

If two providers click "Accept" at the exact same millisecond, the database executes the atomic version check. The first transaction increments the version from `0` to `1`. The second transaction checks `WHERE version = 0`, finds 0 affected rows, and aborts with an `OptimisticLockingFailureException`, which the Global Exception Handler translates into a clean `409 Conflict` response to the second provider.

6. Comprehensive Lifecycle State Machine

A service booking moves through six operational stages before settlement:

State Validation Rules:

- **`PENDING`**: Initial state; visible to all eligible nearby providers via STOMP alerts.
- **`ACCEPTED`**: Exclusive lock acquired by one provider. Removed from all other providers' alert feeds.
- **`ON_THE_WAY`**: Technician is traveling; map displays live coordinate tracking.
- **`IN_PROGRESS`**: Work has commenced; final cost adjustment can be applied by technician.
- **`COMPLETED`**: Invoice generated; customer receives payment prompt.

- `PAID`: Signature verified via Razorpay webhook or confirmed by provider as cash collected. Customer rating and review unlocked.

7. Role-Based Feature Matrix

👤 Customer Portal

- **Dynamic Location Mapping:** Click-to-pin on OpenStreetMap Leaflet map with automatic reverse geocoding via server proxy.
- **Saved Address Book:** Manage multiple tagged addresses (Home, Office, Other).
- **Live Booking Timeline:** WebSocket-driven tracker updating without browser refresh.
- **Payment Choice:** Seamless modal checkout supporting Razorpay Test Gateway (UPI, Cards, NetBanking) and Cash on Delivery.
- **Ratings & Feedback:** Post-service 1-5 star reviews with commentary directly updating the provider's cumulative rating.

🔧 Service Provider Portal

- **Availability Toggle:** Switch status between `ONLINE` and `OFFLINE` with real-time coordinate updates.
- **Real-Time Job Alerts:** Audio-visual modal dialog presenting incoming nearby requests with radial distance and earnings estimate.
- **Job Execution Dashboard:** Stepped status progression buttons (`I am on the way` ➔ `Start Work` ➔ `Complete Work`).
- **Earnings & History:** Aggregate lifetime earnings, completed job history, and customer review summaries.

🛡️ Platform Admin Dashboard

- **Real-Time KPI Cards:** Total platform revenue, active bookings, registered customer count, and provider verification pipeline.
- **Provider Verification Gate:** Inspect provider documents and credentials; approve or reject provider operating licenses.
- **Category & Tariff Management:** Create new service categories, set base inspection fees, and configure hourly rates.
- **Platform Audit & Complaints:** View all system bookings, payment transaction IDs, and customer complaints.

8. Security Architecture & Threat Mitigation

Security Concern	Technical Defense Implemented in HomeServe
Credential Storage	Passwords are never stored in plaintext; salted and hashed using **BCrypt** with strength factor 10.
Session Hijacking	Stateless JWT tokens passed via `Authorization: Bearer <token>` header with expiration time of 24 hours.
Cross-Origin Attacks (CORS)	Strict `CorsConfigurationSource` restricting allowed origins to production domains (`https://*.onrender.com`) and local development endpoints with credentials enabled.
Cross-Site Request Forgery (CSRF)	CSRF disabled because the application uses stateless Bearer tokens and stores no session cookies in the browser.
SQL Injection	Hibernate JPA parameterization using prepared statements and typed JPQL queries. Native SQL queries use positional parameters.
Rate Limiting / Scraping	OpenStreetMap Nominatim queries routed through a Spring Boot backend proxy () with a custom `User-Agent` header, preventing browser cross-origin blocks and client-side IP bans.

9. Production Cloud Deployment Architecture

HomeServe is deployed on **Render Cloud Infrastructure** using an Infrastructure-as-Code Blueprint ():

- **Frontend Service (`homeserve-ui`):** Deployed as a high-performance static site on Render's global CDN with SPA rewrite rules (`/* -> /index.html`) to support deep client-side links.
- **Backend Service (`homeserve-backend`):** Deployed inside an unprivileged Linux container (`appuser`) running Eclipse Temurin 17 JRE. Spring Boot dynamically binds to Render's injected `${PORT}` environment variable.
- **Zero-CORS Multi-Origin Setup:** Backend explicitly permits the deployed frontend origin pattern `https://*.onrender.com` in both Spring Security and STOMP WebSocket configurations.

10. Academic Viva & Project Review Q&A Guide

Prepare for your project presentation with these anticipated technical questions:

Q1: Why did you choose a monolithic architecture instead of microservices?

> **Answer:** "For a hyperlocal on-demand service platform at this scale, a layered monolith provides strong ACID transaction guarantees, zero network serialization latency between services, simplified database migrations via Flyway, and unified deployment. Microservices introduce distributed transaction challenges (like the Saga pattern) and eventual consistency latency during concurrent job dispatch, which is counterproductive when two providers compete for a single job in real time."

Q2: How does the system prevent two providers from accepting the same service request?

> **Answer:** "We implemented Optimistic Concurrency Control using JPA's `@Version` annotation on the `ServiceRequest` entity. When a provider attempts to accept a booking, Hibernate checks if the version in the database matches the version read. The first provider's transaction increments the version number. Any competing simultaneous request finds an outdated version and fails with an `OptimisticLockException`, returning a clean HTTP 409 Conflict to the late provider without locking the entire table."

Q3: How do you calculate distance between customers and service providers?

> **Answer:** "We use the spherical Haversine formula implemented in our Spring Data JPA repository. It takes the customer's latitude and longitude and computes the great-circle distance in kilometers against each provider profile's stored coordinates:
>
> $$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$
>
> d = 2R · arcsin (sin2 (2Δϕ) + cos(ϕ1) cos(ϕ2) sin2 (2Δλ)
>
>)
>
> Providers who are marked `is_online = true`, `is_verified = true`, match the requested service category, and have distance $\leq 25\ \text{km}$≤ 25 km are selected for WebSocket push dispatch."

Q4: Why did you use STOMP over raw WebSockets?

> **Answer:** "Raw WebSockets provide only an unformatted byte/text stream. STOMP (Simple Text Oriented Messaging Protocol) introduces a standardized framing protocol on top of WebSockets with headers, commands (CONNECT, SUBSCRIBE, SEND), and message routing semantics (`/topic` for broadcast and `/queue` for point-to-point). This allows our Spring Boot backend to push targeted notifications to specific providers without implementing custom socket packet parsing."

Q5: Why was a backend proxy needed for OpenStreetMap reverse geocoding?

> **Answer:** "OpenStreetMap's Nominatim API requires all clients to supply an identifiable HTTP `User-Agent` header for abuse monitoring and enforces strict cross-origin policies. Web standards prohibit browser JavaScript from modifying the forbidden `User-Agent` header. By routing geocoding requests through our Spring Boot `GeocodeController`, the backend securely attaches the custom identifier (`HomeServe-Platform/1.0`), avoids browser CORS blocks, and provides a single, controlled gateway."

Document compiled and verified for HomeServe Full-Stack Semester V Project Review.